

Week 4: Your First Machine Learning Model with Scikit-learn

Mentor: Doris Mugah

Session Type: Technical / Hands-On

Level: Beginner Friendly

Learning Objective

By the end of this notebook, you should be able to build and run a simple machine learning model from start to finish using **Scikit-learn**.

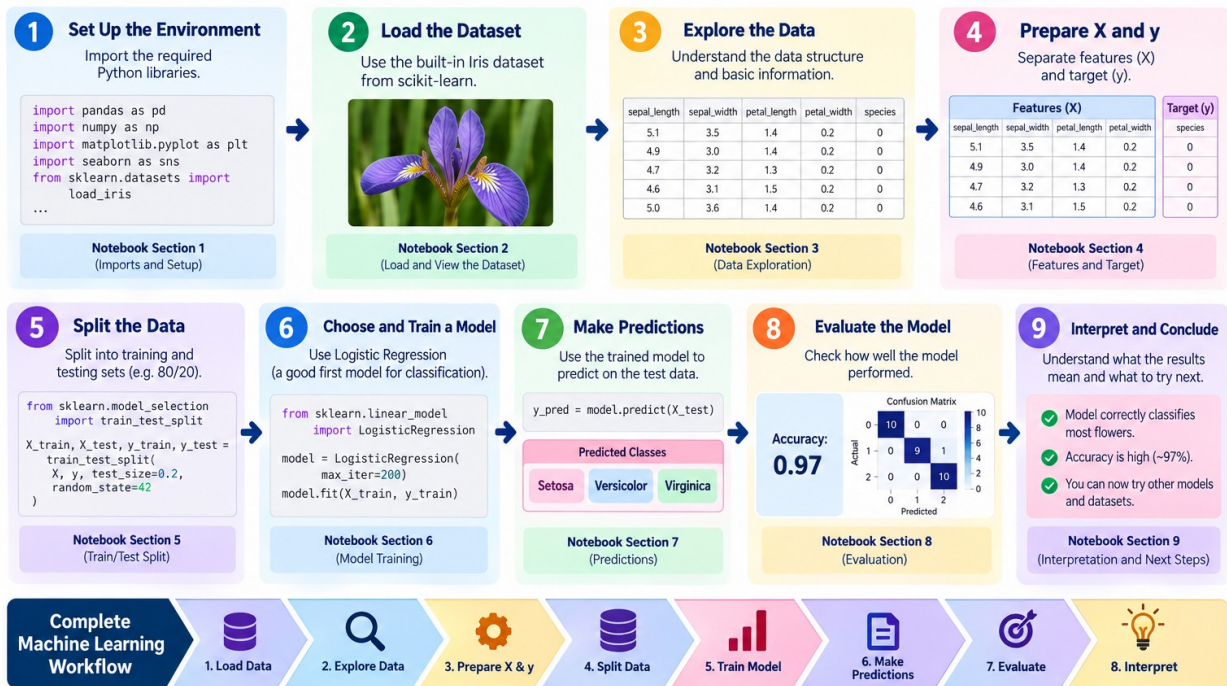
We will learn how to:

1. Understand a simple machine learning problem
2. Load a dataset
3. Explore the data
4. Identify **X (features)** and **y (target)**
5. Split data into training and testing sets
6. Create a classification model
7. Train the model using `fit()`
8. Make predictions using `predict()`
9. Evaluate the model using accuracy
10. Interpret the results

Week 5: First Machine Learning Model with Scikit-learn

Roadmap: From Data to Predictions (Based on the Colab Notebook)

Follow this **step-by-step roadmap** to understand and complete the notebook.



1. What is Machine Learning?

Machine learning allows a computer to learn patterns from existing data and use those patterns to make predictions on new data.

A simple supervised machine learning workflow is:

Dataset → Understand the Problem → X and y → Train/Test Split → Choose Model → Train → Predict → Evaluate → Interpret

In this lesson, we will build a **classification model**.

Classification

Classification is used when the answer we want to predict belongs to a category.

Examples:

- Spam or not spam
- Customer will buy or will not buy
- Fraudulent or legitimate transaction
- Type of flower

For our first model, we will predict the **species of an Iris flower**.

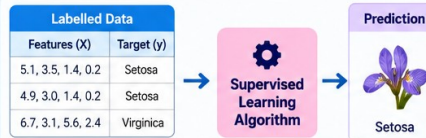
Types of Machine Learning

Three main approaches to learning from data

1. Supervised Learning

Learn from labelled data

The model learns from input data (features) and the correct output (label/target) is already known. It learns the relationship between X (features) and y (target) to make predictions on new data.



Common Algorithms

Classification

(predict a category)

- Logistic Regression
- Decision Trees
- Random Forest
- Support Vector Machine (SVM)
- Naive Bayes
- K-Nearest Neighbors (KNN)
- Gradient Boosting (e.g. XGBoost)

Regression

(predict a number)

- Linear Regression
- Ridge and Lasso Regression
- Decision Trees
- Random Forest Regressor
- Support Vector Regression (SVR)
- Gradient Boosting (e.g. XGBoost)

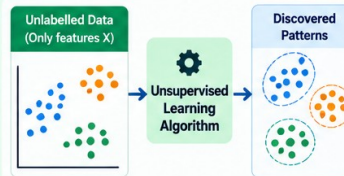
Real-World Examples



2. Unsupervised Learning

Learn from unlabelled data

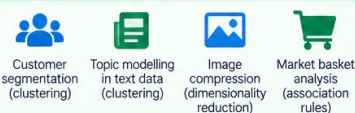
The model works with input data only (no labels) and tries to discover hidden patterns, structure or relationships in the data.



Common Algorithms

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN
- Principal Component Analysis (PCA)
- t-SNE
- Autoencoders
- Association Rule Learning (e.g. Apriori)

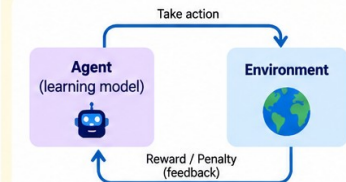
Real-World Examples



3. Reinforcement Learning

Learn by interacting with an environment

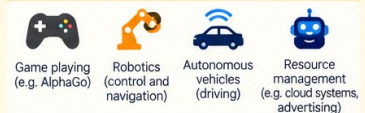
The model (agent) learns by taking actions in an environment and receiving feedback in the form of rewards or penalties. It learns a policy to maximise the cumulative reward over time.



Common Algorithms

- Q-Learning
- Deep Q-Network (DQN)
- Policy Gradient
- Actor-Critic Methods (A2C, A3C)
- Proximal Policy Optimization (PPO)
- Deep Deterministic Policy Gradient (DDPG)

Real-World Examples



2. Import the Libraries

Scikit-learn is already available in Google Colab, so you normally do not need to install it.

We will also use Pandas to view the dataset in a familiar table format.

```
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

print("Libraries imported successfully!")

Libraries imported successfully!
```

3. Load the Iris Dataset

The Iris dataset is included with Scikit-learn.

It contains measurements of flowers from three Iris species:

- Setosa
- Versicolor

- Virginica

The model will learn from flower measurements and try to predict the correct species.

```
iris = load_iris()

print("Dataset loaded successfully!")
print("Target names:", iris.target_names)

Dataset loaded successfully!
Target names: ['setosa' 'versicolor' 'virginica']
```

4. Convert the Data to a Pandas DataFrame

A DataFrame makes the data easier to inspect and understand.

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add the target column
df["target"] = iris.target

# Add readable species names
df["species"] = df["target"].map({
    0: "setosa",
    1: "versicolor",
    2: "virginica"
})

df.tail(10)

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n      \"column\": \"sepal length (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.383695481107378,\n        \"min\": 5.8,\n        \"max\": 6.9,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          6.9,\n          6.5,\n          6.7\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"sepal width (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.26687491868330787,\n        \"min\": 2.5,\n        \"max\": 3.4,\n        \"num_unique_values\": 7,\n        \"samples\": [\n          3.1,\n          2.7,\n          2.5\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"petal length (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.3056868404829434,\n        \"min\": 5.0,\n        \"max\": 5.9,\n        \"num_unique_values\": 7,\n        \"samples\": [\n          5.6,\n          5.1,\n          5.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"petal width (cm)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.24517567397911055,\n        \"min\":
```

```
1.8,\n          \"max\": 2.5,\n          \"num_unique_values\": 6,\n\"samples\": [\n          2.4,\n          2.3,\n          1.8\n],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n}\n    },\n    {\n        \"column\": \"target\",\n        \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 0,\n            \"min\": 2,\n            \"max\": 2,\n            \"num_unique_values\": 1,\n            \"samples\": [\n                2\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        },\n        {\n            \"column\": \"species\",\n            \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\": 1,\n                \"samples\": [\n                    \"virginica\"\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n            }\n        }\n    }\n],\n\"type\": \"dataframe\"}
```

Beginner Check

Look at the first five rows.

Each **row** represents one flower.

The measurement columns are the information we can give the model.

The **species** is what we ultimately want the model to identify.

5. Explore the Dataset

Before training a model, always understand the data you are working with.

```
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
```

Dataset shape: (150, 6)

Column names:

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)', 'target', 'species']
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 150 entries, 0 to 149
```

```
Data columns (total 6 columns):
```

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	target	150 non-null	int64
5	species	150 non-null	object

```
dtypes: float64(4), int64(1), object(1)
```

```
memory usage: 7.2+ KB
```

```
df.describe()
```

```
{  
  "summary": "  
    \"name\": \"df\",  
    \"rows\": 8,  
    \"fields\": [  
      \"column\": \"sepal length (cm)\",  
      \"properties\": {  
        \"dtype\": \"number\",  
        \"std\": 51.24711349471842,  
        \"min\": 0.8280661279778629,  
        \"max\": 150.0,  
        \"num_unique_values\": 8,  
        \"samples\": [  
          5.843333333333334,  
          5.8,  
          150.0  
        ],  
        \"semantic_type\": \"\",  
        \"description\": \"\"  
      },  
      \"column\": \"sepal width (cm)\",  
      \"properties\": {  
        \"dtype\": \"number\",  
        \"std\": 52.08617800869866,  
        \"min\": 0.435866284936698,  
        \"max\": 150.0,  
        \"num_unique_values\": 8,  
        \"samples\": [  
          3.0573333333333337,  
          3.0,  
          150.0  
        ],  
        \"semantic_type\": \"\",  
        \"description\": \"\"  
      },  
      \"column\": \"petal length (cm)\",  
      \"properties\": {  
        \"dtype\": \"number\",  
        \"std\": 51.83521261418364,  
        \"min\": 1.0,  
        \"max\": 150.0,  
        \"num_unique_values\": 8,  
        \"samples\": [  
          3.7580000000000005,  
          4.35,  
          150.0  
        ],  
        \"semantic_type\": \"\",  
        \"description\": \"\"  
      },  
      \"column\": \"petal width (cm)\",  
      \"properties\": {  
        \"dtype\": \"number\",  
        \"std\": 52.636648242617504,  
        \"min\": 0.1,  
        \"max\": 150.0,  
        \"num_unique_values\": 8,  
        \"samples\": [  
          1.1993333333333336,  
          1.3,  
          150.0  
        ],  
        \"semantic_type\": \"\",  
        \"description\": \"\"  
      },  
      \"column\": \"target\",  
      \"properties\": {  
        \"dtype\": \"number\",  
        \"std\": 52.69404575122032,  
        \"min\": 0.0,  
        \"max\": 150.0,  
        \"num_unique_values\": 5,  
        \"samples\": [  
          1.0,  
          2.0,  
          0.8192319205190405  
        ],  
        \"semantic_type\": \"\",  
        \"description\": \"\"  
      }  
    ]  
  },  
  \"type\": \"dataframe\"  
}
```

Think About It

Answer these questions:

1. How many rows are in the dataset?
2. How many flower measurement features are available?
3. Are there any missing values?
4. What are the three flower species?

```
# Check for missing values  
df.isnull().sum()
```

```

sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target              0
species             0
dtype: int64

# See how many flowers belong to each species
df["species"].value_counts()

species
setosa      50
versicolor  50
virginica   50
Name: count, dtype: int64

```

6. Understanding X and y

This is one of the most important concepts in supervised machine learning.

X = Features

X contains the information the model uses to make a prediction.

In this dataset, the features are:

- sepal length
- sepal width
- petal length
- petal width

y = Target

y contains the answer we want the model to learn to predict.

Here, the target is the flower species.

```

# X contains the input features
X = df[iris.feature_names]

# y contains the target
y = df["target"]

print("X shape:", X.shape)
print("y shape:", y.shape)

X shape: (150, 4)
y shape: (150,)

```

```
# View the first few rows of X
X.head()

{"summary":{"\n  \"name\": \"X\",\n  \"rows\": 150,\n  \"fields\": [\n  {\n    \"column\": \"sepal length (cm)\",\n    \"properties\": {\n      \"dtype\": \"number\",\n      \"std\": 0.8280661279778629,\n      \"min\": 4.3,\n      \"max\": 7.9,\n      \"num_unique_values\": 35,\n      \"samples\": [\n        6.2,\n        4.5,\n        5.6\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    \"column\": \"sepal width (cm)\",\n    \"properties\": {\n      \"dtype\": \"number\",\n      \"std\": 0.435866284936698,\n      \"min\": 2.0,\n      \"max\": 4.4,\n      \"num_unique_values\": 23,\n      \"samples\": [\n        2.3,\n        4.0,\n        3.5\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    \"column\": \"petal length (cm)\",\n    \"properties\": {\n      \"dtype\": \"number\",\n      \"std\": 1.7652982332594667,\n      \"min\": 1.0,\n      \"max\": 6.9,\n      \"num_unique_values\": 43,\n      \"samples\": [\n        6.7,\n        3.8,\n        3.7\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    \"column\": \"petal width (cm)\",\n    \"properties\": {\n      \"dtype\": \"number\",\n      \"std\": 0.7622376689603465,\n      \"min\": 0.1,\n      \"max\": 2.5,\n      \"num_unique_values\": 22,\n      \"samples\": [\n        0.2,\n        1.2,\n        1.3\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    }\n  ]\n},\n\"type\": \"dataframe\", \"variable_name\": \"X\"}

# View the first few values of y
y.tail(10)

140    2
141    2
142    2
143    2
144    2
145    2
146    2
147    2
148    2
149    2
Name: target, dtype: int64
```

7. Split the Data into Training and Testing Sets

We should not train and test the model using exactly the same data.

We divide the dataset into:

- **Training data** — used by the model to learn

- **Testing data** — used to check performance on unseen examples

We will use **80% for training** and **20% for testing**.

`random_state=42` helps us get the same split each time we run the notebook.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training targets:", y_train.shape)
print("Testing targets:", y_test.shape)
```

```
Training features: (120, 4)
Testing features: (30, 4)
Training targets: (120,)
Testing targets: (30,)
```

Check Your Understanding

If the dataset contains 150 rows and we use 20% for testing:

- Approximately how many rows are used for training?
- Approximately how many rows are used for testing?

8. Create Our First Model

We will use **Logistic Regression**.

Despite the word *regression* in its name, Logistic Regression is commonly used for **classification**.

For now, focus on the workflow rather than the mathematics behind the algorithm.

```
model = LogisticRegression(max_iter=200)
print(model)
LogisticRegression(max_iter=200)
```

9. Train the Model with `fit()`

`fit()` means:

Learn patterns from the training data.

We give the model:

- `X_train` — the training features
- `y_train` — the correct answers

```
model.fit(X_train, y_train)

print("Model training complete!")

Model training complete!
```

10. Make Predictions with `predict()`

Now we give the trained model the testing features.

These are examples that were not used to train the model.

```
predictions = model.predict(X_test)

print("Predictions:")
print(predictions)

Predictions:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]
```

11. Compare Predictions with the Correct Answers

Let's place the actual and predicted values next to each other.

```
results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": predictions
})

results["Actual Species"] = results["Actual"].map({
    0: "setosa",
    1: "versicolor",
    2: "virginica"
})

results["Predicted Species"] = results["Predicted"].map({
    0: "setosa",
    1: "versicolor",
    2: "virginica"
})

results.head(10)

{"summary": "{\n  \"name\": \"results\",\n  \"rows\": 30,\n  \"fields\": [\n    {\n      \"column\": \"Actual\",
```

```

{"properties": {"dtype": "number", "std": 0, "min": 0, "max": 2, "num_unique_values": 3, "samples": [0, 2, 1], "semantic_type": "", "description": ""}, {"column": "Predicted", "properties": {"dtype": "number", "std": 0, "min": 0, "max": 2, "num_unique_values": 3, "samples": [0, 2, 1], "semantic_type": "", "description": ""}, {"column": "Actual Species", "properties": {"dtype": "category", "num_unique_values": 3, "samples": ["setosa", "virginica", "versicolor"], "semantic_type": "", "description": ""}, {"column": "Predicted Species", "properties": {"dtype": "category", "num_unique_values": 3, "samples": ["setosa", "virginica", "versicolor"], "semantic_type": "", "description": ""}]}
{"type": "dataframe", "variable_name": "results"}

```

12. Evaluate the Model

For this beginner classification exercise, we will use **accuracy**.

Accuracy tells us the proportion of test examples the model predicted correctly.

For example, an accuracy of 0.90 means the model correctly classified about **90%** of the test examples.

```

accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
print(f"Accuracy percentage: {accuracy * 100:.2f}%")

```

```

Accuracy: 0.9666666666666667
Accuracy percentage: 96.67%

```

13. Interpretation

Do not stop after printing a number. Explain what the result means.

Complete this sentence:

The model achieved an accuracy of ____%. This means that it correctly predicted approximately ____% of the flower species in the test data.

Important

A high accuracy on this small educational dataset does **not** mean that every machine learning model will perform this well in real-world situations.

Real datasets can contain:

- Missing values
- Incorrect values
- Imbalanced classes
- Many features
- Noise
- Outliers
- More complicated relationships

```
new_flower = pd.DataFrame(  
    [[5.1, 3.5, 1.4, 0.2]],  
    columns=iris.feature_names  
)  
  
new_prediction = model.predict(new_flower)  
predicted_species = iris.target_names[new_prediction[0]]  
print("Predicted species:", predicted_species)  
Predicted species: setosa
```

Your Turn

Change the flower measurements below and run the cell again.

Observe whether the predicted species changes.

```
# Change these four numbers and run the cell  
  
my_flower = pd.DataFrame(  
    [[6.0, 3.0, 4.5, 1.5]],  
    columns=iris.feature_names  
)  
  
my_prediction = model.predict(my_flower)  
print("Predicted class:", my_prediction[0])  
print("Predicted species:", iris.target_names[my_prediction[0]])  
Predicted class: 1  
Predicted species: versicolor
```